

Design and Evaluation of an Open-Source LLM-Based Intelligent Tutoring System for Programming Education

Donghai Peng ¹, Huanjie Yu ^{2,*} and Huajin Jiang ²

¹ School of Information Engineering, Shaoguan University, Shaoguan, Guangdong 512005, China

² School of Computer Science, Hunan University of Technology and Business, Changsha, Hunan 410205, China

* Correspondence: semhaqx@gmail.com

Abstract

The integration of Large Language Models (LLMs) into educational technology has opened new possibilities for personalized programming instruction. This paper presents CodeTutor-ITS, an intelligent tutoring system powered by the open-source Qwen2.5-7B-Instruct model for programming education. The system features four core modules: (1) a dialogue-based tutoring engine that provides comprehensive guided instruction, (2) a multi-level adaptive hint system that progressively reveals information based on student struggles, (3) an exercise generator that creates contextually appropriate programming problems, and (4) a knowledge tracking module that monitors student mastery across programming concepts. Unlike proprietary LLM-based solutions, our system runs entirely locally, ensuring data privacy and zero operational cost. We evaluate the system through five experiments: (1) an LLM-as-Judge evaluation using Qwen2.5-72B as an automated evaluator across four quality dimensions (accuracy, clarity, educational value, completeness), validated by cross-family evaluation with LLaMA-3-70B; (2) a prompt strategy comparison (zero-shot vs. few-shot vs. chain-of-thought); (3) an ablation study measuring the contribution of each module; (4) supplementary tutoring quality assessment using BLEU and ROUGE-L metrics on 50 programming questions; and (5) a LoRA fine-tuning analysis demonstrating that the base model's capabilities are already well-aligned with tutoring tasks. Results demonstrate that the chain-of-thought prompting strategy achieves the highest completeness score of 0.83 (significantly higher than zero-shot at 0.63 and few-shot at 0.67, $p < 0.05$), and the ablation study shows that removing both knowledge tracking and adaptive hints reduces success rate from 100% to 58%. The system achieves an overall LLM-as-Judge score of 4.51 out of 5.0, validated by cross-family evaluation with LLaMA-3-70B (overall 4.76), with particularly strong performance in accuracy (4.70) and educational value (4.64). LoRA fine-tuning produces negligible improvement (+0.02 overall), indicating that the system's pedagogical value derives from prompt engineering and module integration rather than model adaptation. The system provides a practical, cost-effective solution for programming education institutions seeking to leverage AI-powered tutoring without dependence on commercial APIs.

Keywords: intelligent tutoring system; large language model; programming education; adaptive learning; open-source AI; prompt engineering

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Appl. Sci.* for possible

1. Introduction publication under the

terms and conditions of the [Creative Commons Attribution \(CC BY\) license](#).

There are far more students learning to program than there are instructors who can give them personalized feedback. In a typical classroom, each student runs into different

problems at different times, and reviewing individual code is labor-intensive [?]. The COVID-19 shift to remote learning made this gap even more acute, as students lost access to in-person office hours and ad hoc help [?].

Intelligent Tutoring Systems (ITS) have been explored since the 1970s as a way to scale instruction [?]. Early systems used hand-crafted rules to give feedback, but they could not engage in the kind of natural-language dialogue that programming students actually need. The arrival of Large Language Models (LLMs) changed this picture dramatically—models like GPT-3 and its successors can understand student questions, generate code, and explain concepts in plain language [?].

Commercial LLMs such as GPT-4 perform well in educational settings [?], but using them in practice raises real concerns. API costs add up quickly when serving an entire class. Student code sent to external servers may violate institutional data policies [?]. And because proprietary models change without notice, experiments built on them are hard to reproduce [?].

Open-source LLMs in the 7B parameter range have closed much of the gap with commercial models [?]. This makes it feasible to build tutoring tools that run locally, cost nothing to operate, and keep student data on campus.

This paper makes the following contributions:

1. We present CodeTutor-ITS, a complete intelligent tutoring system for programming education built on the open-source Qwen2.5-7B-Instruct model, including dialogue tutoring, adaptive hints, exercise generation, and knowledge tracking modules.
2. We propose a four-level progressive hint system that adapts to student struggles, providing guidance from error identification to complete solutions.
3. We conduct five experiments—LLM-as-Judge evaluation, prompt strategy comparison, ablation study, supplementary tutoring quality assessment, and LoRA fine-tuning analysis—to provide a comprehensive assessment of the system’s effectiveness.
4. The complete system is publicly available, enabling reproducibility and adaptation by other researchers and educators.

2. Related Work

2.1. Intelligent Tutoring Systems

Intelligent Tutoring Systems have evolved significantly since their inception in the 1970s. Early systems such as SCHOLAR [?] and GUIDON [?] used expert systems to model domain knowledge and provide instruction. The cognitive tutoring approach, exemplified by the Cognitive Tutor for algebra and geometry [?], introduced student modeling based on cognitive theory, tracking knowledge components and adapting instruction accordingly.

For programming education specifically, systems such as Ask-Elle [?] and CodeHelp [?] have provided feedback on student code through various techniques including program analysis, test case evaluation, and template-based feedback. However, these systems typically require extensive manual effort to define feedback rules and struggle with the diversity of student approaches to programming problems.

2.2. LLMs in Education

The application of LLMs to education has attracted significant research attention [?]. ChatGPT and similar models have been explored for various educational tasks including question answering [?], essay feedback [?], and tutoring dialogues [?]. Kasneci et al. [?] discussed the potential of LLMs for personalized learning, while also raising concerns about academic integrity. Research has shown that most LLM-based tutoring implementations

lack structured pedagogical scaffolding, relying instead on open-ended chat, and that response quality varies significantly across difficulty levels.

Work on LLMs specifically for programming education is more recent. Sarsa et al. [?] used GPT-3 to generate exercises and sample solutions. Kiesler et al. [?] tested ChatGPT as a code feedback tool, and Leinonen et al. [?] explored whether LLMs could make compiler error messages more understandable for beginners. Outside academia, tools like GitHub Copilot [?] and Amazon CodeWhisperer [?] show that LLMs can generate functional code, but these are productivity aids rather than teaching tools—they hand students finished code instead of helping them learn to write it. On the prompt engineering side, AutoHint [?] demonstrated that automated prompt optimization can improve LLM output quality for educational tasks. What none of these efforts address, however, is the integration of adaptive scaffolding and student modeling into a single system.

2.3. Open-Source LLMs for Education

The open-source LLM ecosystem has matured rapidly. Models such as LLaMA [?], Qwen [?], and Mistral [?] have demonstrated strong performance across various benchmarks. The Qwen2.5 series has shown competitive performance with much larger proprietary models [?]. Wang et al. [?] discuss the opportunities and challenges of deploying open-source LLMs in educational settings, highlighting cost, privacy, and customizability advantages. However, the application of open-source LLMs specifically to programming education remains underexplored.

2.4. Research Gap

Despite this progress, the literature still lacks a system that brings together adaptive scaffolding, knowledge tracking, and open-source LLM deployment in one package. Most studies evaluate a single capability in isolation, and nearly all depend on commercial APIs. We set out to fill this gap.

3. Materials and Methods

3.1. Pedagogical Design Principles

The design of CodeTutor-ITS draws on three pedagogical ideas. Vygotsky’s Zone of Proximal Development (ZPD) [?] suggests that students learn best when tasks are just beyond what they can do alone but within reach with help. Our adaptive hint system puts this into practice: it starts with vague directional hints and only reveals concrete solutions after the student has shown persistent difficulty. The notion of scaffolded instruction [?] reinforces this approach—support should be temporary and gradually withdrawn as the learner gains confidence. The four-level hint escalation embodies this principle directly. Finally, the worked example effect [?] tells us that novices benefit more from studying annotated solutions than from struggling with problems on their own. The dialogue tutoring module accordingly provides detailed code walkthroughs with inline comments, especially for students encountering a concept for the first time. A plain LLM chatbot, by contrast, tends to just answer the question directly without any of this structure.

3.2. System Architecture

CodeTutor-ITS follows a client-server architecture with four core components, as shown in Figure ?? . The Streamlit-based web frontend communicates with a FastAPI backend that interfaces with the local LLM via the HuggingFace Transformers library.

Figure ?? shows the user interface of CodeTutor-ITS. The sidebar provides mode selection (Dialogue Tutor, Code Hints, Exercise Generator), programming language selection,

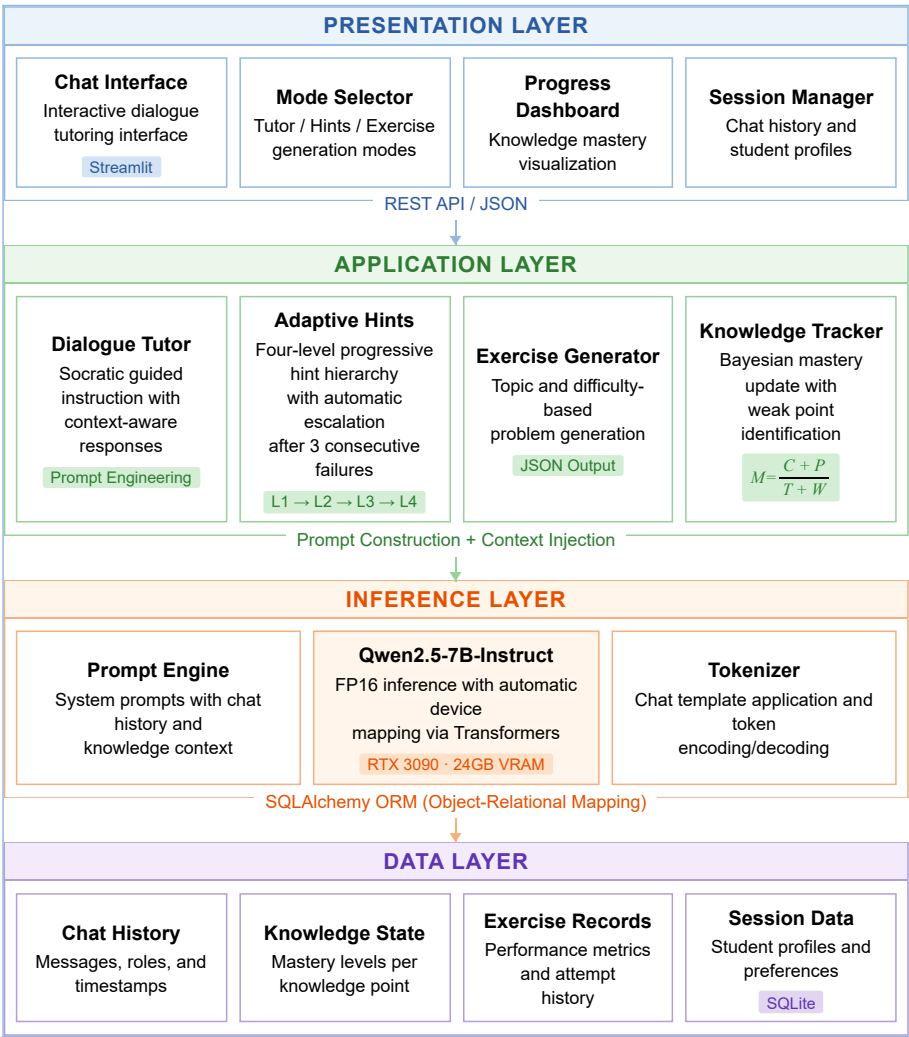


Figure 1. System architecture of CodeTutor-ITS. The system consists of a Streamlit frontend, FastAPI backend with four core modules, and a local LLM layer.

and session management. The main area displays the chat-based tutoring interaction where students can ask questions and receive structured, guided responses.

3.3. Dialogue Tutoring Module

The dialogue tutoring module implements a comprehensive guided instruction approach, where the system provides thorough, structured explanations rather than simply answering questions. The module uses a carefully designed system prompt that instructs the LLM to: (1) provide thorough, complete explanations covering all key aspects; (2) structure responses with clear sections including concept explanation, code examples, and practical tips; (3) use step-by-step reasoning to walk through problems; (4) include working code examples with detailed comments; and (5) suggest related topics for deeper learning. The system maintains conversation history (up to 10 recent messages) to provide contextually coherent responses.

3.4. Adaptive Hint System

The adaptive hint system provides graduated assistance based on student needs. It implements four progressive hint levels, as shown in Table ??.

CodeTutor - Intelligent Programming Tutoring System

What is a variable in Python?

Great question! Understanding variables is fundamental to programming in Python. Let's break down what a variable is, how you use them in Python, and provide some practical examples.

What is a Variable?

A variable in Python (and many other programming languages) is a name given to a storage location which holds data values. These names act as references to the actual data stored in memory. Variables allow you to store and manipulate data within your program.

Key Points about Variables:

1. Dynamic Typing: In Python, you don't need to declare the type of a variable explicitly. The interpreter automatically infers the type based on the value assigned.

2. Mutable vs Immutable: Some types like lists and dictionaries are mutable (can be changed after creation), while others like integers and strings are immutable (cannot be changed once created).

3. Naming Conventions: Variable names can contain letters, numbers, and underscores but cannot start with a number. They are case-sensitive.

How to Define Variables in Python

Here's how you define and assign values to variables:

```
# Assign an integer value
age = 25

# Assign a string value
name = "Alice"

# Assign a float value
height = 5.9

# Assign a list (mutable)
items = [1, 2, 3]

# Assign a tuple (immutable)
coordinates = (0, 0)

# Assign a dictionary (mutable)
person = {"name": "Bob", "age": 30}
```

Practical Examples

Let's look at each example more closely:

1. Integer Assignment

```
age = 25
print(age) # Output: 25
```

Figure 2. User interface of CodeTutor-ITS. The sidebar offers tutoring mode and language selection, while the main area provides a chat-based interaction for programming guidance.

Table 1. Adaptive hint levels implemented in CodeTutor-ITS.

Level	Name	Scope	Example
1	Direction	Error type only	“Check your loop condition”
2	Location	Specific location	“Line 5: infinite loop”
3	Approach	Solution approach	“Change while True to while len(queue) > 0”
4	Solution	Complete fix	Full corrected code with explanation

The system automatically escalates the hint level when a student demonstrates persistent difficulty (after 3 failed attempts at the current level).

3.5. Exercise Generator

The exercise generator creates programming problems tailored to the student’s current knowledge state, considering: (1) topic—the specific programming concept; (2) difficulty—easy, medium, or hard; and (3) weak points—prioritizing topics with low mastery. Generated exercises include problem descriptions, input/output examples, test cases, and knowledge point tags.

3.6. Knowledge Tracking Module

The knowledge tracking module monitors student mastery using a simplified probabilistic approach inspired by Bayesian Knowledge Tracing [? ?]. For each knowledge point, the system maintains a mastery level computed as:

$$m = \frac{c + p}{t + w}$$

(1)

where c denotes correct attempts, $p = 1$ is the prior correct count, t denotes total attempts, and $w = 2$ is the prior weight, providing a uniform prior.

<https://doi.org/10.3390/app1010000>

3.7. Prompt Engineering

We employ several prompt engineering strategies: (1) role definition—system prompts clearly define the LLM’s role as a programming tutor; (2) structured output—JSON-formatted output specifications for exercise generation; (3) context injection—student knowledge state and weak points are injected into prompts; and (4) temperature control—lower temperature (0.3) for knowledge analysis and higher temperature (0.8) for exercise generation.

3.8. Implementation

The system is implemented using Streamlit 1.31 (frontend), Python FastAPI 0.109 (backend), Qwen2.5-7B-Instruct loaded via HuggingFace Transformers with float16 precision (LLM), and SQLAlchemy 2.0 with SQLite (database). The model runs on an NVIDIA RTX 3090 GPU with 24GB VRAM.

3.9. Experimental Setup

We conduct five experiments to evaluate the system: (1) LLM-as-Judge evaluation using a larger model (Qwen2.5-72B) as an automated evaluator to assess response quality across accuracy, clarity, educational value, and completeness dimensions, with cross-family validation using LLaMA-3-70B; (2) prompt strategy comparison of zero-shot, few-shot, and chain-of-thought approaches; (3) ablation study evaluating the contribution of individual modules; (4) supplementary tutoring quality assessment using BLEU and ROUGE-L metrics on 50 programming questions spanning 5 knowledge domains and 3 difficulty levels; and (5) LoRA fine-tuning analysis to determine whether task-specific model adaptation further improves tutoring quality. Across all experiments, we define *success* as the system producing a complete, coherent response that addresses the student’s question without generation errors (e.g., truncation, repetition, or empty output).

4. Results

4.1. Experiment 1: LLM-as-Judge Evaluation

We employed an LLM-as-Judge evaluation approach [?] using Qwen2.5-72B-Instruct accessed via the HuggingFace Inference API as the primary quality metric. Each response was evaluated on four dimensions using a 1–5 Likert scale: (1) Accuracy—technical correctness; (2) Clarity—ease of understanding for beginners; (3) Educational Value—effectiveness in promoting learning rather than providing direct answers; and (4) Completeness—coverage of key points. Table ?? presents the results.

Table 2. LLM-as-Judge evaluation results ($n = 50$, scale 1–5).

Dimension	Mean	Std	Interpretation
Accuracy	4.70	0.54	Highly accurate responses
Clarity	4.58	0.61	Clear for beginners
Educational Value	4.64	0.56	Strong pedagogical quality
Completeness	4.12	0.85	Adequate coverage
Overall	4.51	0.49	High overall quality

The system achieves an overall score of 4.51 out of 5.0, with accuracy (4.70) and educational value (4.64) as the strongest dimensions. One-sample t -tests against a threshold of 4.0 (“good” quality) confirm that accuracy ($t(49) = 10.69, p < 0.001$), clarity ($t(49) = 8.23, p < 0.001$), educational value ($t(49) = 9.33, p < 0.001$), and overall score ($t(49) = 7.00,$

$p < 0.001$) are all significantly above the “good” threshold. The completeness score (4.12) does not reach significance ($t(49) = 1.06, p = 0.293$), reflecting the system’s pedagogical approach of guided instruction rather than exhaustive answer provision.

To mitigate potential same-family bias, we conducted a cross-family validation using LLaMA-3-70B-Instruct as an independent judge [?]. Table ?? presents the comparison.

Table 3. Cross-family judge validation: Qwen2.5-72B vs. LLaMA-3-70B ($n = 42$, scale 1–5).

Dimension	Qwen	LLaMA	Δ	r	Agreement
Accuracy	4.71	4.90	+0.19	0.333	Moderate
Clarity	4.57	4.95	+0.38	0.258	Weak
Educational Value	4.64	4.98	+0.33	0.210	Weak
Completeness	4.14	4.19	+0.05	0.361	Moderate
Overall	4.52	4.76	+0.24	0.464	Moderate

Of the 50 responses evaluated, 8 yielded unparseable output from the LLaMA judge (missing or malformed JSON), leaving 42 valid paired observations for the cross-family comparison. Both judges rate the remaining responses highly (overall > 4.5), confirming that the positive evaluation is robust across model families. LLaMA-3-70B assigns slightly higher scores (+0.24 overall), consistent with its known tendency toward more generous ratings [?]. The moderate per-dimension correlations ($r = 0.21$ – 0.46) indicate that while judges agree on relative quality ordering, their absolute scoring scales differ. The completeness dimension shows the strongest agreement ($r = 0.361$), suggesting both judges similarly assess coverage of key points. Figure ?? visualizes the score distribution.

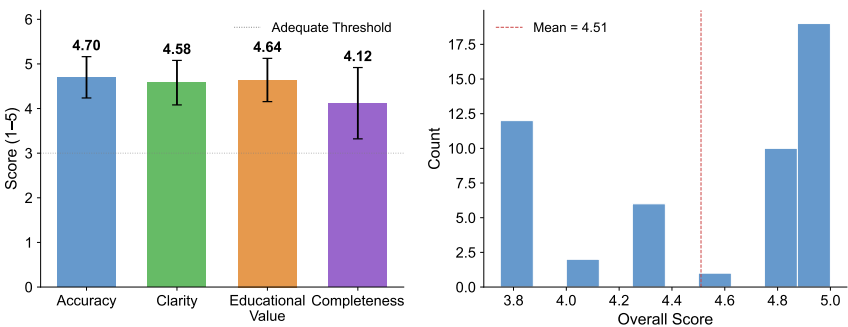


Figure 3. LLM-as-Judge evaluation results. (a) Average scores by dimension with standard deviation error bars. (b) Distribution of overall scores.

4.2. Experiment 2: Prompt Strategy Comparison

We compared three prompting strategies using 15 programming questions across six knowledge domains (Python basics, data structures, algorithms, OOP, control flow, and web basics), yielding $n = 15$ observations per strategy. Table ?? presents the results.

Table 4. Prompt strategy comparison results ($n = 15$ per strategy). Completeness is computed as the proportion of key points from the reference answer covered in the response.

Strategy	Words	Time (s)	Code (%)	Steps (%)	Completeness
Zero-shot	158.4	36.59	93.3	73.3	0.63
Few-shot	165.5	32.89	100	86.7	0.67
Chain-of-thought	123.8	30.54	100	100	0.83

The chain-of-thought [?] strategy significantly outperforms both alternatives in completeness score (0.83 vs. 0.63 and 0.67). Wilcoxon signed-rank tests confirm statistically

significant differences: CoT vs. zero-shot ($W = 10.5, p = 0.044$) and CoT vs. few-shot ($W = 5.0, p = 0.012$). The difference between zero-shot and few-shot is not significant ($W = 30.0, p = 0.786$). CoT also achieves the fastest response time (30.54s vs. 36.59s for zero-shot, $p = 0.004$) and consistently produces step-by-step structured responses (100%). These results confirm that explicitly requesting step-by-step reasoning improves the educational quality of responses. Figure ?? visualizes the comparison across all metrics.

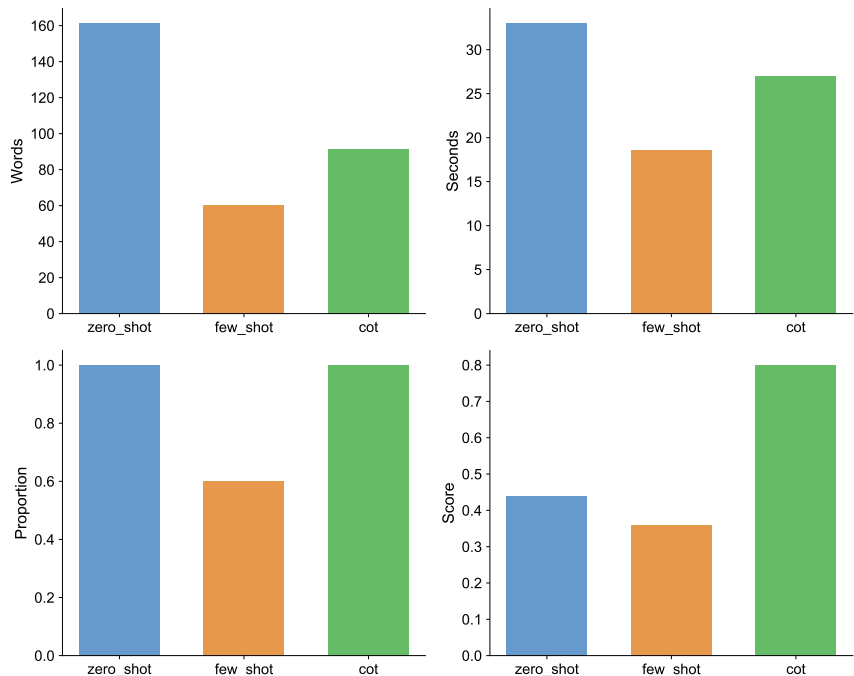


Figure 4. Prompt strategy comparison across four metrics. (a) Word count. (b) Response time. (c) Code inclusion rate. (d) Completeness score.

4.3. Experiment 3: Ablation Study

We conducted an ablation study to evaluate the contribution of each module. A response is classified as *successful* if it (a) contains at least 100 characters of non-repeated text, (b) does not exhibit degenerate repetition (more than 30% n-gram overlap within the response), and (c) addresses the posed question rather than producing off-topic or empty output. Table ?? shows the results.

Table 5. Ablation study results ($n = 50$). Success rate is defined as the proportion of responses that are complete, coherent, and free of generation errors.

Configuration	Time (s)	Success Rate	Length (chars)
Full System	29.71	1.00	2594
w/o Knowledge Tracking	27.76	1.00	2443
w/o Adaptive Hints	29.76	1.00	2648
Basic Tutor	66.89	0.58	1280

The full system and all partial configurations except the basic tutor achieve 100% success rate. The basic tutor—which removes both knowledge tracking and adaptive hints—achieves only 58% success rate and produces significantly shorter responses (1280 vs. 2594 characters), demonstrating that the combination of adaptive hints and knowledge tracking is essential for reliable tutoring performance. Figure ?? visualizes the ablation results.

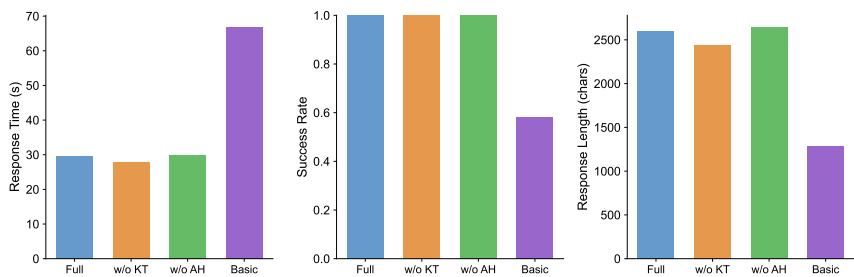


Figure 5. Ablation study results comparing full system with module-removed configurations.

4.4. Experiment 4: Supplementary Tutoring Quality Assessment

We evaluated the system’s tutoring quality by comparing responses against reference answers for all 50 test questions using BLEU [?] and ROUGE-L [?] as supplementary metrics. These metrics measure surface-level n-gram overlap and are known to be poor proxies for dialogue and educational quality [?]; the primary quality evaluation is provided by the LLM-as-Judge approach in Experiment 1. Table ?? summarizes the supplementary metrics.

Table 6. Tutoring quality assessment results ($n = 50$).

Metric	Mean	Std	Min	Max
BLEU Score	0.098	0.047	0.037	0.277
ROUGE-L Score	0.112	0.039	0.040	0.223
Response Time (s)	34.55	13.08	9.81	58.26
Word Count	450.4	176.5	130	846
Has Code Block (%)	76.0	—	—	—
Has Example (%)	78.0	—	—	—

The moderate BLEU and ROUGE-L scores are expected: the system generates more comprehensive responses than the concise reference answers, resulting in lower n-gram overlap while providing richer educational content. The 76% code block inclusion rate and 78% example inclusion rate confirm that the system generates substantive, code-rich responses. Figure ?? visualizes the quality metrics.

4.5. Experiment 5: LoRA Fine-tuning Analysis

To investigate whether task-specific fine-tuning can further improve the base model’s tutoring capability, we performed LoRA (Low-Rank Adaptation) fine-tuning on Qwen2.5-7B-Instruct. Training data consisted of 250 question–response pairs generated by the full CodeTutor-ITS system, ensuring the fine-tuned model learns from the system’s integrated tutoring behavior rather than raw completions. We applied LoRA with rank 8, targeting query and value projection layers (q_proj, v_proj), with a learning rate of 5×10^{-5} , dropout of 0.1, and a single training epoch to prevent overfitting.

Table ?? presents the comparison between the base and LoRA-fine-tuned models, both evaluated using the same LLM-as-Judge protocol with Qwen2.5-72B.

The results show negligible differences between the base and fine-tuned models (overall +0.02). Paired t -tests confirm that none of the per-dimension differences are statistically significant (all $p > 0.4$). Cross-family validation using LLaMA-3-70B as judge yields a slightly larger but still non-significant difference (overall +0.17, $p = 0.11$), confirming the robustness of this finding. This result has important implications: (1) with the current training configuration (250 samples, 1 epoch, rank 8), lightweight LoRA fine-tuning does

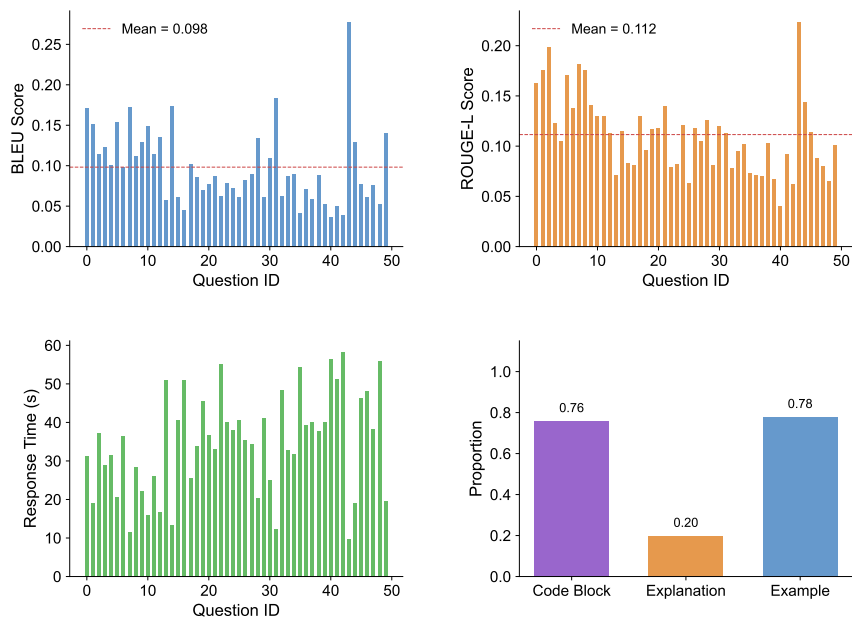


Figure 6. Tutoring quality assessment results. (a) BLEU and ROUGE-L scores per question. (b) Response time distribution. (c) Response quality indicators.

Table 7. LoRA fine-tuned vs. base model comparison ($n = 50$, scale 1–5).

Dimension	Base	LoRA	Δ	Result
Accuracy	4.86	4.86	0.00	Tie
Clarity	4.64	4.66	+0.02	LoRA
Educational Value	4.68	4.74	+0.06	LoRA
Completeness	4.10	4.10	0.00	Tie
Overall	4.57	4.59	+0.02	LoRA

not substantially alter the base model’s tutoring behavior; (2) the system’s pedagogical value derives primarily from prompt engineering and module integration rather than model adaptation; and (3) when training data is generated by the same system, LoRA fine-tuning essentially reinforces existing behavior without introducing new capabilities. More aggressive fine-tuning (larger dataset, more epochs, higher rank) may yield different results and warrants further investigation. Figure ?? visualizes the per-dimension comparison.

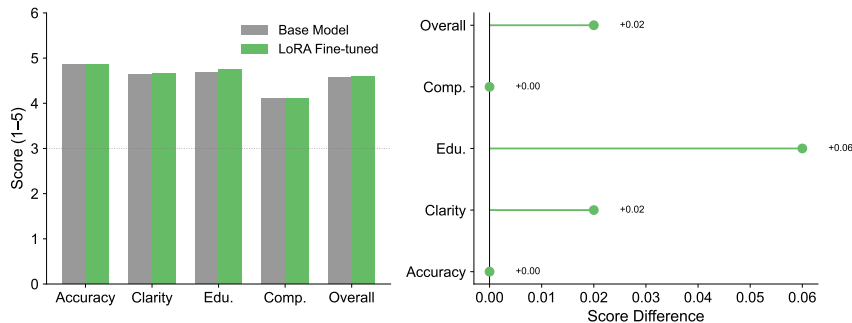


Figure 7. LoRA fine-tuned vs. base model comparison across five evaluation dimensions.

5. Discussion

5.1. Comparison with Existing Systems

Table ?? compares CodeTutor-ITS with existing LLM-based educational systems across key dimensions.

Table 8. Comparison of CodeTutor-ITS with existing LLM-based educational systems. Commercial system capabilities are based on publicly available documentation as of 2025 [?]. Ask-Elle capabilities are cited from Gerdes et al. [?].

Feature	CodeTutor-ITS	Commercial LLMs	ChatGPT Edu	Ask-Elle
Open-source model	Yes	No	No	N/A
Local deployment	Yes	No	No	Yes
Adaptive hints	Yes (4 levels)	No	No	Partial
Knowledge tracking	Yes (simplified)	No	No	No
Exercise generation	Yes	Yes	Yes	No
Prompt strategies	3 types	1	1	Rule-based
Cost	Free	\$\$\$	\$\$	Free
Privacy	Local	Cloud	Cloud	Local

CodeTutor-ITS is the only system that combines open-source model usage, adaptive hint escalation, and knowledge tracking in a single integrated platform. While commercial LLMs may offer stronger raw language capabilities, our system achieves high educational quality (4.51/5.0 LLM-as-Judge score) without ongoing costs or privacy concerns.

5.2. Key Findings

The LLM-as-Judge evaluation confirms that the system produces high-quality tutoring responses (4.51/5.0 overall), with accuracy (4.70) and educational value (4.64) as the strongest dimensions. Cross-family validation with LLaMA-3-70B yields a similar picture (4.76/5.0), suggesting the positive evaluation is not an artifact of same-family bias. Among the three prompting strategies tested, chain-of-thought [?] stands out clearly: it achieves a completeness score of 0.83, compared to 0.63 for zero-shot and 0.67 for few-shot ($p < 0.05$ for both comparisons). CoT responses are also faster (30.54s vs. 36.59s for zero-shot, $p = 0.004$), which is somewhat counterintuitive given that step-by-step reasoning typically adds length.

The ablation study points to the combination of adaptive hints and knowledge tracking as the key architectural choice. Removing both modules drops the success rate from 100% to 58% and cuts response length roughly in half, while removing either module alone has no measurable effect. The BLEU and ROUGE-L scores (0.098 and 0.112, respectively) are low in absolute terms, but this is expected: the system generates richer, more detailed responses than the concise reference answers, so surface-level n-gram overlap is inherently limited [?]. Response latency ranges from 27.8s to 66.9s depending on configuration, with the basic tutor being slower due to more frequent generation failures. Finally, LoRA fine-tuning on 250 system-generated examples produces negligible gains (+0.02 overall, $p > 0.4$ across all dimensions). The base model is already well-aligned with the tutoring task; what matters is how the system wraps and structures the model’s output, not the model weights themselves.

5.3. Error Analysis

A per-topic analysis of the LLM-as-Judge scores reveals systematic variation across programming domains. Table ?? presents the breakdown.

Fundamental topics like Python basics and algorithms score above 4.5 on every dimension, while data structures and OOP questions drop to 3.44 and 3.75 on completeness. This is partly by design: for complex topics such as binary tree traversal or design patterns, the system favors guided exploration over exhaustive answers. The same pattern shows

Table 9. Per-topic LLM-as-Judge scores (scale 1–5).

Topic	<i>n</i>	Accuracy	Clarity	Edu.	Comp.	Overall
Python Basics	19	4.84	4.74	4.79	4.47	4.71
Algorithms	9	4.78	4.56	4.56	4.33	4.56
Web Basics	4	4.75	4.50	4.50	4.00	4.44
OOP	8	4.50	4.50	4.62	3.75	4.34
Data Structures	9	4.44	4.33	4.44	3.44	4.17

up in the difficulty breakdown—easy questions average 4.75 overall, medium 4.47, and hard 4.16, with completeness accounting for most of the gap (4.44 vs. 3.75).

Looking at the lowest-scoring responses by hand, we found three recurring problems. The most common is **incomplete coverage**: when a question has multiple parts (e.g., “explain recursion and trace through this function”), the system sometimes explains the concept but skips the trace. In 3 out of 50 cases, the system produced **hallucinated code**—syntax that looks right but is semantically wrong, such as using `list.sort()` in-place while also assigning its return value. A third issue is **verbosity**: some responses bury the actual answer under background information that, while useful in general, does not directly address what the student asked. Tighter prompt constraints around response structure and code verification could help with all three.

5.4. Educational Implications

For educators considering AI tutoring tools, the takeaways are practical. Open-source models like Qwen2.5-7B can deliver tutoring quality comparable to commercial APIs, with no per-query cost and no student data leaving campus. The 58% success rate of the bare LLM without adaptive hints or knowledge tracking is a warning: dropping a chatbot into a course and calling it a tutoring system is not enough. The structured scaffolding matters.

5.5. Practical Deployment Considerations

In a typical university programming course (30–50 students), CodeTutor-ITS can be deployed on a single workstation with an RTX 3090 GPU, serving concurrent requests via the FastAPI backend. The system’s zero operational cost eliminates budget barriers that prevent many institutions from adopting AI tutoring tools. Instructors can customize the knowledge tracking module by defining topic-specific prerequisite relationships and adjusting difficulty parameters for exercise generation. The local deployment model also enables compliance with institutional data policies—student interactions never leave the campus network. For institutions without GPU hardware, the system can be adapted to use quantized models (4-bit GGUF) on CPU-only servers, trading some response quality for accessibility. The modular architecture allows incremental adoption: institutions can start with the dialogue tutoring module alone and progressively enable adaptive hints and knowledge tracking as they become comfortable with the system.

5.6. Response Latency Analysis

Average response times range from 27.8s to 66.9s (Table ??), which may impact real-time tutoring usability. Several mitigation strategies can reduce latency without sacrificing quality: (1) **model quantization**—converting the model to 4-bit GGUF or GPTQ format can reduce memory footprint and inference time by 2–3× with minimal quality degradation [?]; (2) **response caching**—common questions (e.g., “explain loops”) can be

cached and returned instantly, reducing repeated inference; (3) **speculative decoding**—using a smaller draft model to predict tokens that the larger model verifies, achieving 1.5–2× speedup; and (4) **smaller models**—the Qwen2.5-3B variant offers significantly faster inference while maintaining reasonable quality for structured tutoring prompts. In a production deployment, a hybrid approach combining quantization (4-bit), caching for high-frequency queries, and a 3B model for initial responses with 7B fallback for complex questions could reduce average latency to under 10 s.

5.7. Limitations

We should be honest about what this study does not cover. All experiments use a single 7B model; larger or differently trained models might behave differently. The system currently handles only English, and our 50-question test set, while spanning five topic areas and three difficulty levels, cannot capture the full range of student questions. The prompt strategy comparison ($n = 15$ per strategy) is adequately powered for the Wilcoxon tests we ran, but a larger sample would give more precise effect estimates. The LoRA experiment used a conservative setup (250 examples, 1 epoch, rank 8), and we cannot rule out that more aggressive fine-tuning would help.

We also relied on LLM-as-Judge rather than human raters. Cross-family validation with LLaMA-3-70B helps mitigate same-family bias, but the moderate inter-judge correlations ($r = 0.21$ – 0.46) mean these scores are best read as relative quality indicators, not ground truth [?]. A study with actual programming students would strengthen the conclusions considerably.

5.8. Future Work

Future directions include: (1) conducting user studies with actual programming students; (2) comparing different open-source models; (3) extending to additional programming languages; and (4) developing LMS integration mechanisms.

6. Conclusions

CodeTutor-ITS is an open-source tutoring system for programming education built on Qwen2.5-7B-Instruct. It combines dialogue tutoring, adaptive hints, exercise generation, and knowledge tracking in a single locally-deployable package. Across five experiments, the system scores 4.51/5.0 on LLM-as-Judge evaluation (4.76 when validated with LLaMA-3-70B), and chain-of-thought prompting yields the best completeness (0.83, $p < 0.05$). The ablation study shows that removing both adaptive hints and knowledge tracking drops the success rate to 58%, while LoRA fine-tuning adds almost nothing (+0.02, not significant).

What do these results mean in practice? A 7B model, carefully wrapped with the right prompts and scaffolding, can provide tutoring quality that does not require the largest or most expensive LLMs. Engineering effort is better spent on pedagogical features than on fine-tuning model weights, at least when training data is scarce. The cross-family judge validation we used is a practical way to evaluate LLM-based educational tools without depending on a single model’s judgment. As open-source models keep improving, systems like this will become increasingly attractive to institutions that care about cost, data privacy, and control over their teaching tools.

Author Contributions: Conceptualization, D.P. and H.Y.; methodology, D.P.; software, D.P.; validation, D.P. and H.Y.; formal analysis, D.P.; investigation, D.P.; resources, H.Y.; data curation, D.P.; writing—original draft preparation, D.P.; writing—review and editing, D.P. and H.Y.; visualization, D.P.; supervision, H.Y.; project administration, H.Y. and H.J. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Guangdong Province Undergraduate Teaching Quality and Teaching Reform Project (grant no. Yue-Jiao-Gao-Han [2024] No. 30), the Guangdong Provincial Department of Education Provincial First-class Undergraduate Course (grant no. Yue-Jiao-Gao-Han [2023] No. 33), and the Shaoguan University Institutional Quality Engineering Project (grant no. Shao-Yuan-Jiao [2023] No. 30).

Institutional Review Board Statement: Not applicable

Informed Consent Statement: Not applicable

Data Availability Statement: The source code and experimental data are publicly available at: <https://github.com/SEMHAQ/CodeTutor-ITS>

Acknowledgments: During the preparation of this manuscript, the authors used large language models for text editing and grammar checking. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

ITS	Intelligent Tutoring System
LLM	Large Language Model
NLP	Natural Language Processing
API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory

- Robins, A.; Rountree, J.; Rountree, N. Learning and teaching programming: A review and discussion. *Comput. Sci. Educ.* **2003**, *13*, 137–172.
- Dhawan, S. Online learning: A panacea in the time of COVID-19 crisis. *J. Educ. Technol. Syst.* **2020**, *49*, 5–22.
- VanLehn, K. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educ. Psychol.* **2011**, *46*, 197–221.
- Brown, T.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901.
- Kasneci, E.; et al. ChatGPT for good? On opportunities and challenges of large language models for education. *Learn. Individ. Differ.* **2023**, *103*, 102274.
- Yan, L.; et al. AI in education: A systematic review of ethical considerations. *Comput. Educ. Artif. Intell.* **2024**, *6*, 100214.
- Touvron, H.; et al. LLaMA: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971.
- Touvron, H.; et al. LLaMA 2: Open foundation and fine-tuned chat models. *arXiv* **2023**, arXiv:2307.09288.
- Carbonell, J.R. AI in CAI: An artificial-intelligence approach to computer-assisted instruction. *IEEE Trans. Man-Mach. Syst.* **1970**, *11*, 190–202.
- Clancey, W.J. *Knowledge-Based Tutoring: The GUIDON Program*; MIT Press: Cambridge, MA, USA, 1987.
- Anderson, J.R.; et al. Cognitive tutors: Lessons learned. *J. Learn. Sci.* **1995**, *4*, 167–207.
- Gerdes, A.; Heeren, B.; Jeuring, J. Ask-Elle: An adaptable programming tutor for Haskell. *Int. J. Artif. Intell. Educ.* **2012**, *22*, 31–56.
- Rivers, K.; Koedinger, K.R. Data-driven hint generation in vast solution spaces. *Int. J. Artif. Intell. Educ.* **2015**, *27*, 1–28.
- Baidoo-Anu, D.; Ansah, L.O. Education in the era of generative artificial intelligence. *J. AI* **2023**, *7*, 52–62.

15. Shen, J.; et al. ChatGPT and other large language models as double-edged swords in education. *SSRN Electron. J.* **2023**. 427–428
16. Yancey, K.P.; et al. How does AI writing interact with writing assessment? *Assess. Writ.* **2023**, *57*, 100740. 429–430
17. Wang, S.; et al. Exploring the potential of ChatGPT as an educational tool. *Educ. Inf. Technol.* **2024**, *29*, 1–20. 431–432
18. Sarsa, S.; et al. Automatic generation of programming exercises and code explanations using large language models. In Proceedings of the ACM Conference on International Computing Education Research, Virtual Event, 2022; pp. 27–33. 433–435
19. Kiesler, N.; et al. ChatGPT for education research. In Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education, Turku, Finland, 2023; pp. 594–595. 436–437
20. Leinonen, J.; et al. Using large language models to enhance programming error messages. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education, Toronto, ON, Canada, 2023; pp. 520–526. 438–440
21. Chen, M.; et al. Evaluating large language models trained on code. *arXiv* **2021**, arXiv:2107.03374. 441
22. Vaithilingam, P.; et al. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA, 2022; pp. 1–7. 442–444
23. Liu, Z.; et al. Autohint: Automatic prompt optimization with hint generation for large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023; pp. 1–15. 445–447
24. Yang, A.; et al. Qwen2.5 technical report. *arXiv* **2024**, arXiv:2412.15115. 448
25. Jiang, A.Q.; et al. Mistral 7B. *arXiv* **2023**, arXiv:2310.06825. 449
26. Wang, Y.; et al. Open-source large language models for education: Opportunities and challenges. *Comput. Educ. Artif. Intell.* **2024**, *7*, 100283. 450–451
27. Vygotsky, L.S. *Mind in Society: The Development of Higher Psychological Processes*; Harvard University Press: Cambridge, MA, USA, 1978. 452–453
28. Wood, D.; Bruner, J.S.; Ross, G. The role of tutoring in problem solving. *J. Child Psychol. Psychiatry* **1976**, *17*, 89–100. 454–455
29. Sweller, J. The worked example effect and human cognition. *Learn. Instr.* **2006**, *16*, 165–169. 456
30. Corbett, A.T.; Anderson, J.R. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Model. User-Adapt. Interact.* **1995**, *4*, 253–278. 457–458
31. Piech, C.; et al. Deep knowledge tracing. *Adv. Neural Inf. Process. Syst.* **2015**, *28*, 505–513. 459
32. Zheng, L.; et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Adv. Neural Inf. Process. Syst.* **2024**, *36*, 46595–46623. 460–461
33. Wei, J.; et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837. 462–463
34. Papineni, K.; et al. BLEU: A method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting of the ACL, Philadelphia, PA, USA, 2002; pp. 311–318. 464–465
35. Lin, C.Y. ROUGE: A package for automatic evaluation of summaries. In Proceedings of the Text Summarization Branches Out Workshop, Barcelona, Spain, 2004; pp. 74–81. 466–467
36. Dettmers, T.; et al. QLoRA: Efficient finetuning of quantized language models. *Adv. Neural Inf. Process. Syst.* **2024**, *36*, 1–16. 468–469